

EEL4783 Final Project (Part 1/2): High-Level Synthesis of a Sobel Edge Detection Filter

Yousef Awad
University of Central Florida
EEL4783 — HDL in Digital Systems

April 16, 2026

Introduction

This report documents the High-Level Synthesis (HLS) of a Sobel Edge Detection filter using Vivado HLS 2019.1. The design processes a 256×256 grayscale image and produces an edge-detected output using the Sobel operator. The filter applies 3×3 horizontal (G_x) and vertical (G_y) convolution kernels and computes the gradient magnitude $|\nabla I| \approx |G_x| + |G_y|$, clamped to $[0, 255]$.

The design was synthesized with default settings (no HLS pragmas) targeting a Zynq-7020 (xc7z020c1g484-1) with a 10 ns clock period.

1 Question 1: Synthesis Results

1.1 1a) Default Synthesis Metrics

The design was synthesized using default Vivado HLS scheduling with no optimization pragmas applied. Table 1 summarizes the key metrics from the C-synthesis report.

Table 1: Default Synthesis Results for `sobel_filter`

Metric	Value
Target Clock Period	10.00 ns
Estimated Clock Period	8.253 ns
Clock Uncertainty	1.25 ns
Worst-Case Latency (cycles)	2,359,809
Best-Case Latency (cycles)	131,585
Pipeline Type	None
DSP48E Used	2
FFs Used	295
LUTs Used	687
BRAM_18K Used	0
URAM Used	0

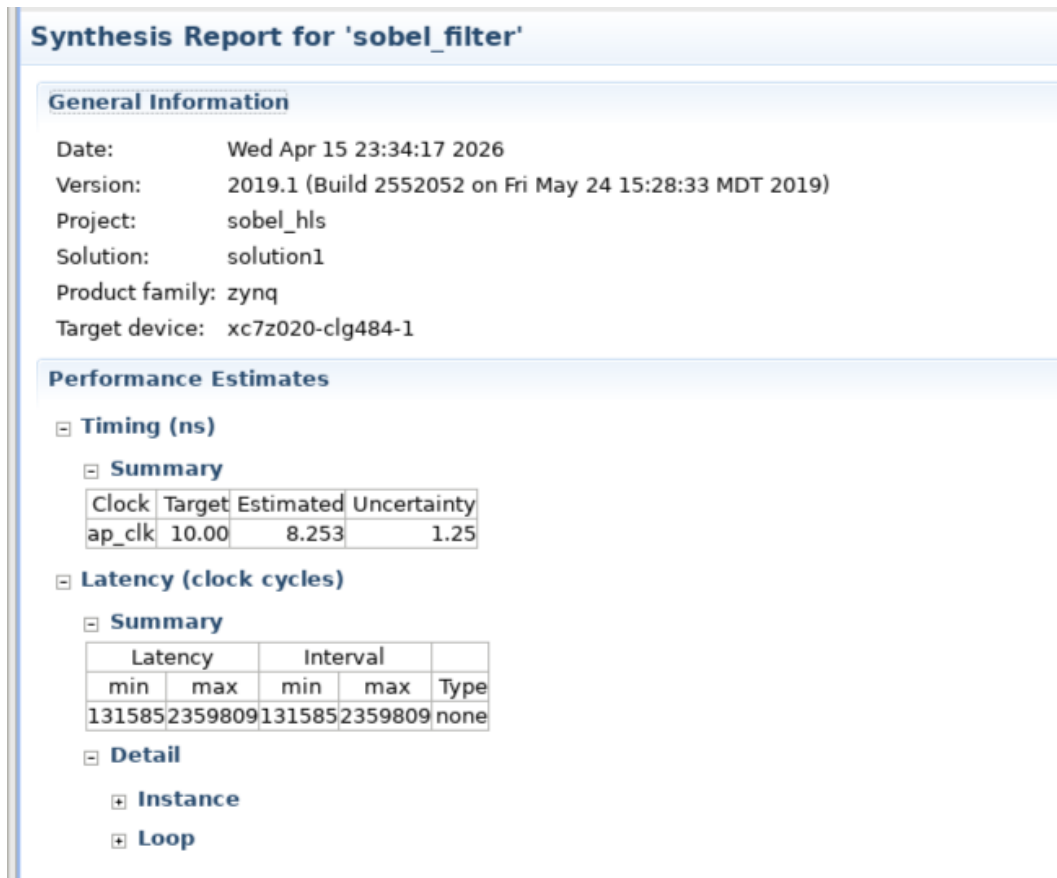


Figure 1: Vivado HLS Synthesis Summary for `sobel_filter`, showing timing, latency, and utilization estimates.

The estimated clock period of 8.253 ns is well below the 10 ns target, confirming the design meets timing with 1.747 ns of positive slack. The design is not pipelined (0 out of 4 loops pipelined), which explains the high worst-case latency of 2,359,809 cycles—each of the $256 \times 256 = 65,536$ inner pixels must be processed sequentially through the 3×3 kernel convolution.

The latency variance between best-case (131,585) and worst-case (2,359,809) arises from the conditional border-pixel check: border pixels skip the kernel convolution entirely (2 cycles), while interior pixels execute the full nested kernel loop (up to 36 cycles per pixel).

The two DSP48E blocks are bound to multiply-accumulate (MAC) operations ($i0 + i1 * i2$) within the kernel loop, where each source pixel is multiplied by its corresponding Sobel coefficient. The 295 FFs absorb the FSM state encoding, loop counters (`x_0_reg`, `y_0_reg`, `kx_0_reg`, `ky_0_reg`), partial sum accumulators (`sum_x`, `sum_y`), and pipeline registers. The 687 LUTs cover address generation, comparison logic, absolute-value select muxes, and the magnitude addition.

1.2 1b) Analysis Perspective

The Analysis Perspective provides a cycle-level schedule view of how operations are mapped to FSM states.

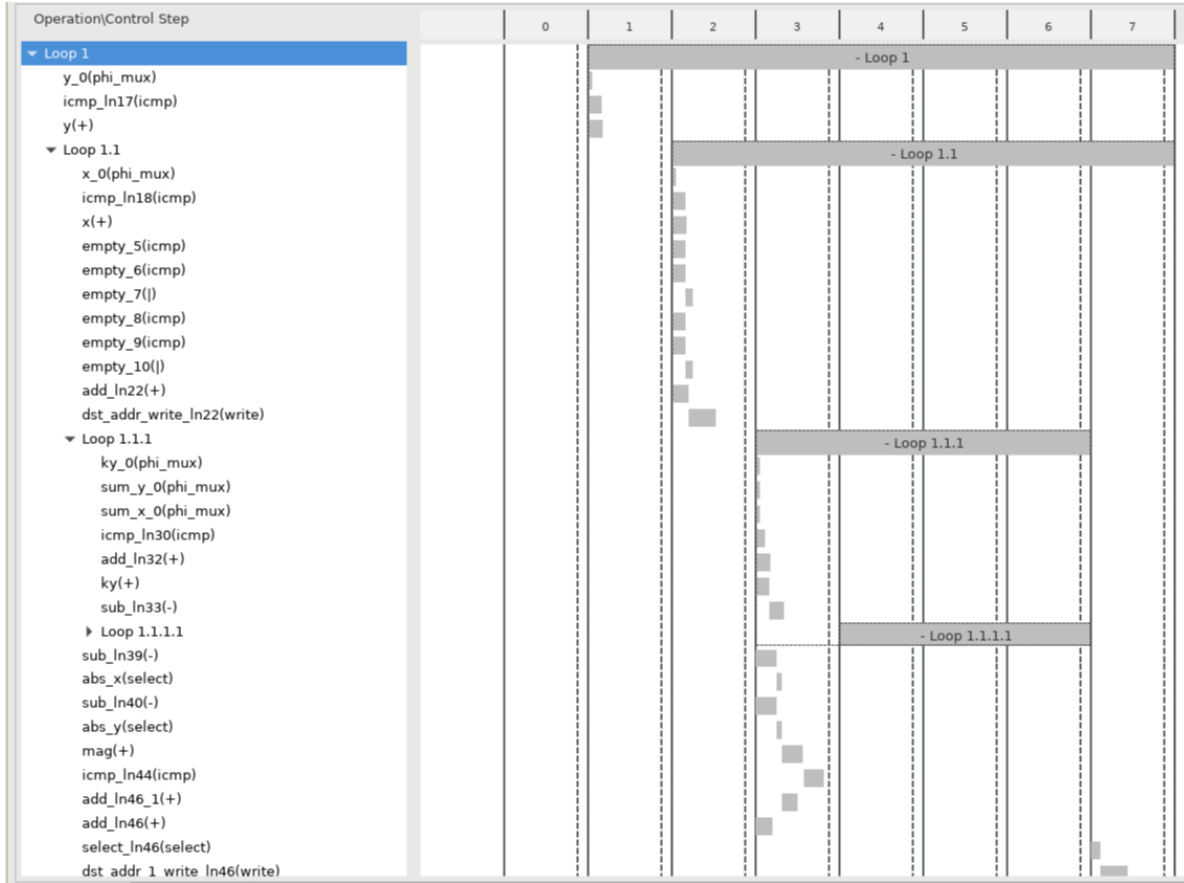


Figure 2: Analysis Perspective showing the schedule of operations across FSM states in `sobel_filter`.

The schedule viewer reveals how each iteration of the innermost kernel loop requires multiple FSM states for: memory reads from the `src` array (via the `ap_memory` interface), multiply-accumulate operations using the Sobel coefficients accessed from the `Gx` and `Gy` distributed ROMs, and the final gradient magnitude computation with clamped write to `dst`.

The innermost loop (**Loop 1.1.1.1**) has a latency of 3 cycles per iteration with a trip count of 3, giving 9 cycles per kernel-row pass. The kernel-column loop (**Loop 1.1.1**) wraps this with latency 11 cycles $\times$ 3 iterations = 33 cycles for the full 3×3 convolution per pixel.

1.3 1c) Resource Profile

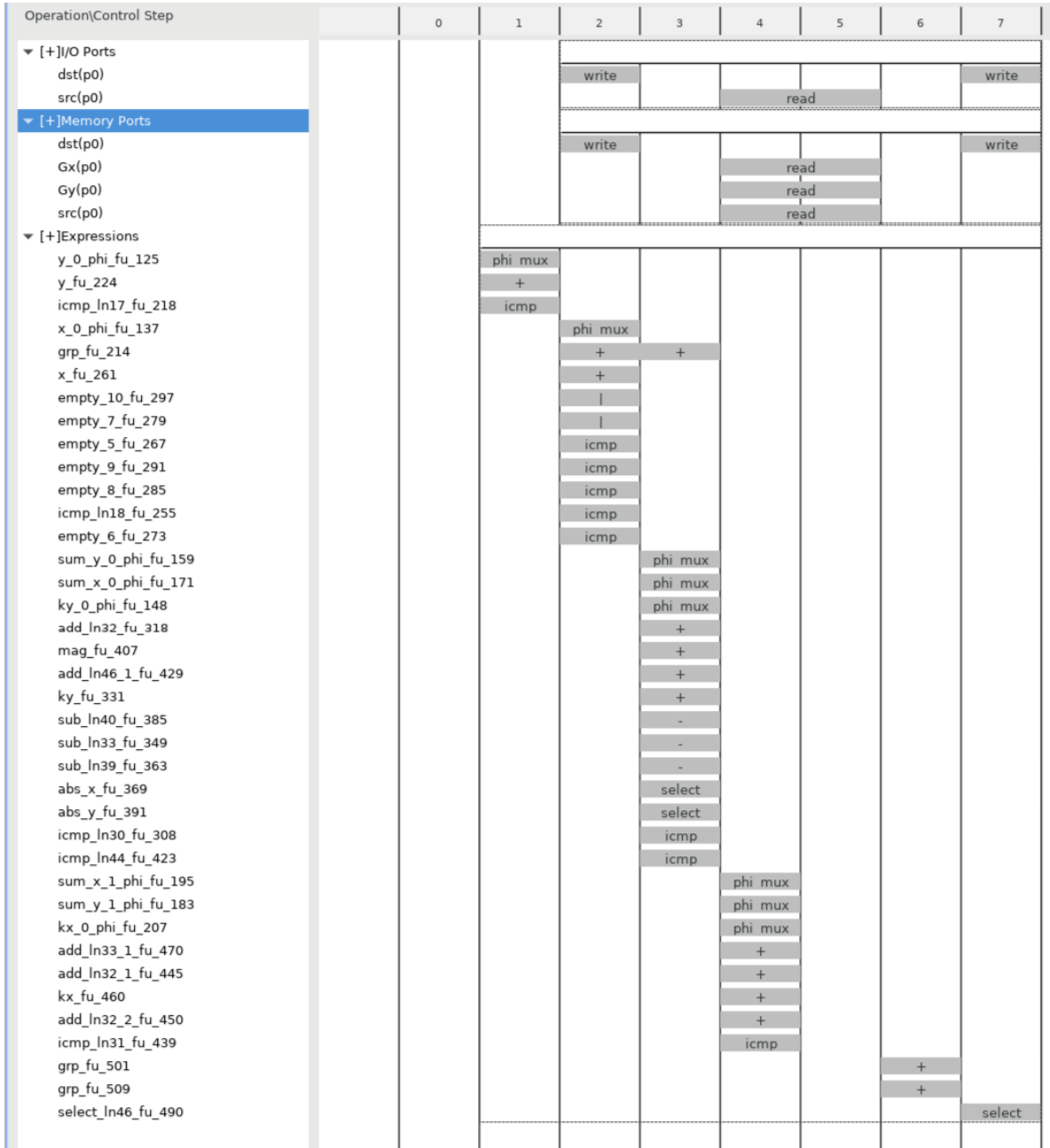


Figure 3: Resource Profile showing DSP48E allocation and memory bindings within `sobel_filter`.

The Bind Op Report confirms both DSP48E blocks are allocated to the `sobel_filter` module for the MAC operations (`sobel_filter_mac_muladd_8ns_3s_32ns_32.1.1`), each computing `pixel * coefficient + accumulator` within the inner product loop.

The Bind Storage Report shows the two Sobel kernels implemented as small distributed ROMs:

- `Gx_U` (`sobel_filter_Gx`): 9 words $\times$ 3 bits, using 3 FFs + 1 LUT
- `Gy_U` (`sobel_filter_Gy`): 9 words $\times$ 3 bits, using 3 FFs + 1 LUT

No BRAM or URAM is used. The input/output arrays are accessed through external `ap_memory` interfaces with 16-bit addresses (sufficient for $2^{16} = 65,536$ elements), 8-bit data for `src_q0` (read), and 8-bit data for `dst_d0` (write).

2 Question 2: C/RTL Co-simulation

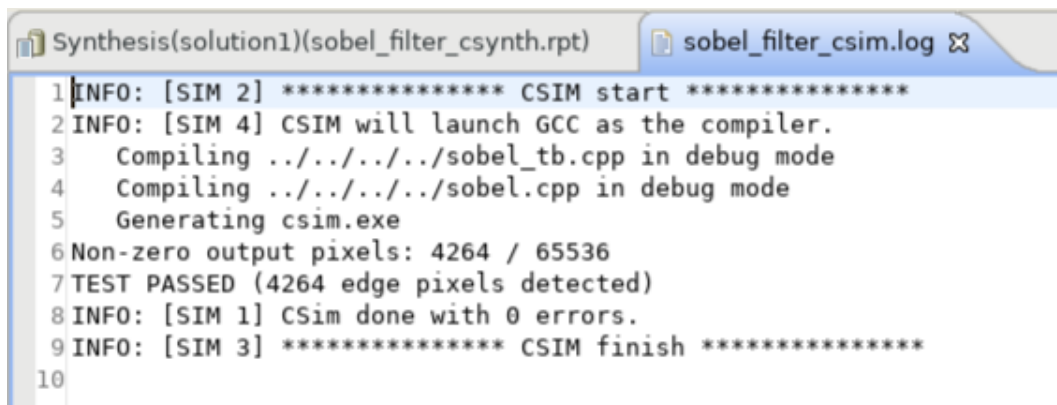
2.1 2a) Co-simulation Report

C-simulation was first run to verify functional correctness. The testbench generates a synthetic 256×256 image with vertical stripe patterns (period 32 pixels, alternating between intensity 200 and 50) and a horizontal bar (intensity 255), applies the Sobel filter, and compares every output pixel against a software reference implementation.

The C-simulation output confirms zero mismatches:

```
Non-zero output pixels: 4264 / 65536
TEST PASSED (4264 edge pixels detected)
```

C/RTL co-simulation was then run using the synthesized Verilog model under XSIM. The same testbench stimulus is fed into the RTL, and outputs are compared against the C model.



```
Synthesis(solution1)(sobel_filter_csynth.rpt)  sobel_filter_csim.log
1 INFO: [SIM 2] ***** CSIM start *****
2 INFO: [SIM 4] CSIM will launch GCC as the compiler.
3   Compiling ../../../../sobel_tb.cpp in debug mode
4   Compiling ../../../../sobel.cpp in debug mode
5   Generating csim.exe
6 Non-zero output pixels: 4264 / 65536
7 TEST PASSED (4264 edge pixels detected)
8 INFO: [SIM 1] CSim done with 0 errors.
9 INFO: [SIM 3] ***** CSIM finish *****
10
```

Figure 4: C/RTL co-simulation report showing PASS status with measured latency and interval.

The co-simulation completed with status **PASS**, confirming the synthesized RTL is functionally equivalent to the C model. The simulation log reports:

```
RTL Simulation : 1 / 1 [100.00%] @ "23251435000"
$finish called at time : 23251475 ns
```

The measured RTL latency of approximately 23.25 ms (at the 10 ns clock period) corresponds to $\sim 2,325,143$ cycles, consistent with the synthesis estimate of 2,359,809 worst-case cycles. The small difference arises because the actual test image has some border pixels that take fewer cycles than the worst case.

Since only one transaction was simulated, the initiation interval (II) is reported as N/A. For this non-pipelined design, the next input can be accepted after latency + 1 clock cycles.

2.2 2b) Waveform Trace Analysis

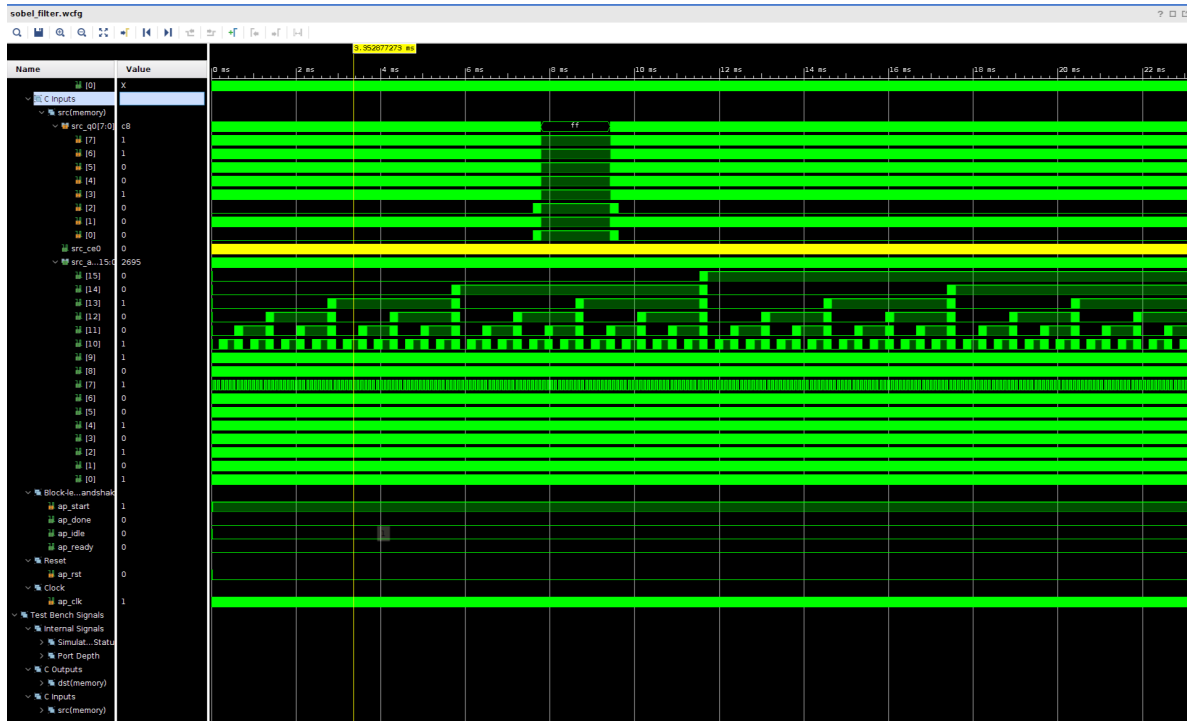


Figure 5: Co-simulation waveform showing the `ap_ctrl_hs` handshake signals and memory interface activity.

The waveform trace confirms the standard `ap_ctrl_hs` block-level protocol:

- `ap_start` is asserted by the testbench to initiate image processing.
- The design enters the active computation phase, sequentially reading pixels from `src` and writing edge-detected results to `dst`.
- `ap_done` pulses high once all 65,536 output pixels have been written.
- `ap_ready` signals that the design can accept a new input transaction.
- The `src_address0` and `dst_address0` signals show the sequential memory access pattern, sweeping through the image row by row.

The total active computation window visible in the trace spans the full ~ 23.25 ms latency, dominated by the un-pipelined sequential traversal of all image pixels and their 3×3 neighborhoods.

2.3 2c) Export RTL

The C-synthesis step generates both VHDL and Verilog RTL under the solution directory:

- `sobel_hls/solution1/syn/verilog/` — Verilog sources
- `sobel_hls/solution1/syn/vhdl/` — VHDL sources

The generated RTL consists of:

- **sobel_filter.v**: Top-level module containing the 8-state FSM (`ap_CS_fsm`), loop counters (`y_0_reg`, `x_0_reg`, `ky_0_reg`, `kx_0_reg`), partial-sum accumulators, address generation logic, multiplexers, and instantiations of the DSP-based MAC and ROM primitives.
- **sobel_filter_Gx.v** / **sobel_filter_Gy.v**: Distributed ROM modules storing the Sobel kernel coefficients as $9\text{-entry} \times 3\text{-bit}$ lookup tables. Implemented using FFs and LUTs rather than BRAM, appropriate for such small constant arrays.
- **sobel_filter_mac_muladd_8ns_3s_32ns_32_1_1.v**: The multiply-accumulate primitive (`i0 + i1 * i2`) mapped to DSP48E slices. Two instances are used—one for the G_x accumulation and one for the G_y accumulation.

Note: The IP Catalog packaging step (`export_design -format ip_catalog`) fails on Vivado HLS 2019.1 due to a known timestamp overflow bug where `core_revision` (a Unix timestamp) exceeds the 32-bit signed integer range after year 2024. This is a tool bug, not a design issue. The synthesized RTL is fully available and functionally verified in the solution directory.

Closing Notes

The default synthesis run produces a working Sobel Edge Detection filter that correctly processes a 256×256 grayscale image on the Zynq-7020 target. The design achieves an estimated clock frequency of $1/8.253\text{ns} \approx 121\text{MHz}$, uses 2 DSP48E blocks, 295 FFs, and 687 LUTs with zero BRAM—representing less than 2% utilization of the target device.

Without any optimization pragmas, the four nested loops execute sequentially, giving a worst-case latency of 2,359,809 cycles ($\sim 23.6\text{ms}$ per frame, or ~ 42 frames/second). Part 2 of this project will explore the design space by applying HLS directives (loop pipelining, array partitioning, loop unrolling, dataflow) to reduce latency and improve throughput at the cost of additional resource usage.

A Appendix A: HLS C++ Source Code

A.1 A.1 sobel.h

```

1 #ifndef SOBEL_H
2 #define SOBEL_H
3
4 #define WIDTH 256
5 #define HEIGHT 256
6
7 typedef unsigned char pixel_t;
8
9 void sobel_filter(pixel_t src[HEIGHT * WIDTH], pixel_t dst[HEIGHT * WIDTH]);
10
11 #endif // SOBEL_H

```

A.2 A.2 sobel.cpp

```

1 #include "sobel.h"
2
3 // Sobel Edge Detection Filter
4 // Synthesizable with Vivado HLS -- no pragmas (default scheduling)
5
6 void sobel_filter(pixel_t src[HEIGHT * WIDTH], pixel_t dst[HEIGHT * WIDTH]) {
7
8     // Sobel kernels
9     const int Gx[3][3] = {
10         {-1, 0, 1},
11         {-2, 0, 2},
12         {-1, 0, 1}
13     };
14     const int Gy[3][3] = {
15         { 1, 2, 1},
16         { 0, 0, 0},
17         {-1, -2, -1}
18     };
19
20     // Process each pixel (borders are set to 0)
21     for (int y = 0; y < HEIGHT; y++) {
22         for (int x = 0; x < WIDTH; x++) {
23
24             // Border pixels -- no valid 3x3 neighborhood
25             if (y == 0 || y == HEIGHT - 1 || x == 0 || x == WIDTH - 1) {
26                 dst[y * WIDTH + x] = 0;
27                 continue;
28             }
29
30             int sum_x = 0;
31             int sum_y = 0;
32
33             // Apply 3x3 Sobel kernels
34             for (int ky = -1; ky <= 1; ky++) {
35                 for (int kx = -1; kx <= 1; kx++) {
36                     int pixel = (int)src[(y + ky) * WIDTH + (x + kx)];
37                     sum_x += pixel * Gx[ky + 1][kx + 1];
38                     sum_y += pixel * Gy[ky + 1][kx + 1];
39                 }

```



```

40     }
41
42     // Gradient magnitude approximation (avoid sqrt for HLS efficiency)
43     int abs_x = (sum_x < 0) ? -sum_x : sum_x;
44     int abs_y = (sum_y < 0) ? -sum_y : sum_y;
45     int mag = abs_x + abs_y;
46
47     // Clamp to [0, 255]
48     if (mag > 255) mag = 255;
49
50     dst[y * WIDTH + x] = (pixel_t)mag;
51 }
52 }
53 }

```

A.3 A.3 sobel_tb.cpp (Testbench)

```

1  #include <stdio>
2  #include <stdlib>
3  #include <cmath>
4  #include "sobel.h"
5
6  // -----
7  // Software reference Sobel (for comparison with HLS output)
8  // -----
9  static void sobel_reference(pixel_t *src, pixel_t *dst) {
10     const int Gx[3][3] = {{-1,0,1},{-2,0,2},{-1,0,1}};
11     const int Gy[3][3] = {{ 1,2,1},{ 0,0,0},{-1,-2,-1}};
12
13     for (int y = 0; y < HEIGHT; y++) {
14         for (int x = 0; x < WIDTH; x++) {
15             if (y == 0 || y == HEIGHT-1 || x == 0 || x == WIDTH-1) {
16                 dst[y*WIDTH+x] = 0;
17                 continue;
18             }
19             int sx = 0, sy = 0;
20             for (int ky = -1; ky <= 1; ky++)
21                 for (int kx = -1; kx <= 1; kx++) {
22                     int p = (int)src[(y+ky)*WIDTH+(x+kx)];
23                     sx += p * Gx[ky+1][kx+1];
24                     sy += p * Gy[ky+1][kx+1];
25                 }
26             int mag = abs(sx) + abs(sy);
27             if (mag > 255) mag = 255;
28             dst[y*WIDTH+x] = (pixel_t)mag;
29         }
30     }
31 }
32
33 // -----
34 // Generate a synthetic test image with clear edges
35 // -----
36 static void generate_test_image(pixel_t *img) {
37     for (int y = 0; y < HEIGHT; y++) {
38         for (int x = 0; x < WIDTH; x++) {
39             // Vertical stripe pattern -- strong vertical edges
40             if ((x / 32) % 2 == 0)

```

```

41         img[y * WIDTH + x] = 200;
42     else
43         img[y * WIDTH + x] = 50;
44     }
45 }
46 // Add a horizontal bar for horizontal edges
47 for (int y = HEIGHT/3; y < HEIGHT/3 + 20; y++)
48     for (int x = 0; x < WIDTH; x++)
49         img[y * WIDTH + x] = 255;
50 }
51
52 // -----
53 // Main testbench
54 // -----
55 int main() {
56     static pixel_t src[HEIGHT * WIDTH];
57     static pixel_t dst_hls[HEIGHT * WIDTH];
58     static pixel_t dst_ref[HEIGHT * WIDTH];
59
60     // 1. Generate test image
61     generate_test_image(src);
62
63     // 2. Run HLS function under test
64     sobel_filter(src, dst_hls);
65
66     // 3. Run software reference
67     sobel_reference(src, dst_ref);
68
69     // 4. Compare
70     int err_count = 0;
71     for (int i = 0; i < HEIGHT * WIDTH; i++) {
72         if (dst_hls[i] != dst_ref[i]) {
73             if (err_count < 20) { // print first 20 mismatches
74                 int y = i / WIDTH;
75                 int x = i % WIDTH;
76                 printf("MISMATCH at (%d,%d): HLS=%d REF=%d\n",
77                     x, y, dst_hls[i], dst_ref[i]);
78             }
79             err_count++;
80         }
81     }
82
83     // 5. Sanity check -- edges should exist in the output
84     int nonzero = 0;
85     for (int i = 0; i < HEIGHT * WIDTH; i++)
86         if (dst_hls[i] > 0) nonzero++;
87
88     printf("Non-zero output pixels: %d / %d\n", nonzero, HEIGHT * WIDTH);
89
90     if (err_count == 0 && nonzero > 0) {
91         printf("TEST PASSED (%d edge pixels detected)\n", nonzero);
92         return 0;
93     } else {
94         printf("TEST FAILED (mismatches=%d, nonzero=%d)\n", err_count, nonzero);
95         return 1;
96     }
97 }

```